

Python 程序设计基础第五章作业

姓名：路柠泽

学号：2022010076

1. 习题 1

1.1 题目

编写函数 filter_chars(), 删除字符串中的非英文字母。

1.2 题目思路

创建空列表，遍历字符串，将字母添加到列表中，最后将列表内容作为字符串输出。

1.3 实现代码

```
1 def filter_chars(str1):
2     str2=[]
3     for a in str1:
4         if a.isalpha():
5             str2.append(a) #append if is alpha
6     return ''.join(str2) #turn into string and output
7 str1=input()
8 print(filter_chars(str1))
```

1.4 本地测试结果

- Case1-用户输入 this is chapter5_1.py, 输出结果为 thisischapterpy, 正确。

```
this is chapter5_1.py
thisischapterpy
```

- Case2-用户输入 a&b cost \$50!, 输出结果为 abcost, 正确。

```
a&b cost $50!
abcost
```

2. 习题 2

2.1 题目

接受用户输入的四则运算表达式，使用栈进行表达式求值。算术表达式是形如“ $1 - 2 + 3 * 4$ ”这样的表达式。为简单计，不考虑括号。

2.2 题目思路

遍历用户输入的表达式，创建临时变量储存读取到的数和运算符。当读取到下一个运算符或结尾时，进行当前数和运算符的计算：若为加减则将数或其相反数压栈，若为乘除则上一值出栈，并将计算结果压栈；运算符初始值为加。最终将栈内数加和即为所求结果。

2.3 实现代码

```
1 def calculate(s):
2     stack=[]
3     num=0
4     op='+'
5     flag=0 #distinguish between integer and decimal parts
6     for i in range(len(s)):
7         c=s[i]
8         if c=='.':
9             flag+=1
10        if c.isdigit():
11            if not flag:
12                num=int(c)+10*num
13            else:
14                num=(10**(flag-1)*num*10+int(c))/10**flag
15                flag+=1
16        #read a number
17        if(not c.isdigit() and c!=' ' and c!='.') or i==len(s)-1:
18            if op=='+':
19                stack.append(num)
20            elif op=='-':
21                stack.append(-num)
22            elif op=='*':
23                stack.append(stack.pop()*num)
24            elif op=='/':
25                stack.append(stack.pop()/num)
26        #read an operator and use the stack to calculate and storage
27        op=c
28        num=0
29        flag=0
30    return sum(stack)
31 s=input("Please enter an expression: ")
32 print("The result is: ",calculate(s))
```

2.4 本地测试结果

- Case1-用户输入 $1 - 2 + 3 * 4$ ，计算结果为 11，正确。

```
Please enter an expression: 1 - 2 + 3 * 4
The result is: 11
```

- Case2-用户输入 $3.6 * 2 / 3 + 1.1$ ，计算结果为 3.5，正确。

```
Please enter an expression: 3.6 * 2 / 3 + 1.1
The result is: 3.5
```

3. 习题 3

3.1 题目

编写程序，随机生成 4 个 1~10 之间的整数，允许重复，判断其能否算出 24。计算规则为：（1）允许使用加减乘除等运算；（2）允许使用括号改变优先级。

3.2 题目思路

运用函数递归遍历所有的计算可能。在列表中随机选取一对数作为第一步运算（加减乘除），将结果与其他数组成新列表进行递归，能够得到 24（考虑浮点数精度误差）即返回 True。生成随机数列表并进行判断。

3.3 实现代码

```
1  import random
2  def p24(nums):
3      if len(nums)==1:
4          return abs(nums[0]-24)<1e-6
5          #(consider float accuracy)
6      for i in range(len(nums)):
7          a=nums[i]
8          for j in range(1+i,len(nums)):
9              b=nums[j]
10             other=nums[:]
11             other.remove(a)
12             other.remove(b)
13             #pick a random pair for the first step
14             for c in [a+b,a-b,b-a,a*b]:
15                 if p24([c]+other):
16                     return True
17                 if b!=0 and p24([a/b]+other):
18                     return True
19                 if a!=0 and p24([b/a]+other):
20                     return True
21             #recursion with remaining numbers
22     return False
23 nums=[random.randint(1,10)for n in range(4)]
24 #generate random numbers
25 print(nums)
26 if p24(nums):
27     print("Can get 24")
28 else:
29     print("Cannot get 24")
```

3.4 本地测试结果

- Case1- 9, 4, 1, 2 能够得到 24。

```
[9, 4, 1, 2]  
Can get 24
```

- Case2-9, 10, 2, 9 无法得到 24。

```
[9, 10, 2, 9]  
Cannot get 24
```

4. 习题 4

4.1 题目

编写程序，随机生成 13 张扑克牌（无大小王牌），统计其牌点（point）。扑克牌牌张（card）有牌花（suit）与牌阶（rank）两种属性。使用字符串“23456789TJQKA”表示牌阶，每一牌阶用单个字符表示，顺序从小到大，其中“T”表示 10。使用字符串“CDHS”表示牌花，每种牌花用单个字符表示，顺序从小到大，分别为梅花（club）、方块（diamond）、红桃（heart）和黑桃（spade）。牌点规定如下：“A”记为 4 点，“K”记为 3 点，“Q”记为 2 点，“J”记为 1 点，其他牌记为 0 点。如果熟悉桥牌（bridge），编写函数，返回其合理开叫（opening bidding）。叫牌时使用“P”表示通过，“NT”表示无将（notrump），无将大于全部花色。叫牌时，可能需要根据叫牌或定约（contract）情况修正牌点。例如，无将定约时应增加长套牌型点，扣除单张大牌点；有将定约时亦存在类似规则。

4.2 题目思路

创建字符串 suits 和 ranks 储存牌花与牌阶，字典 points 储存牌阶对应点数，生成列表 cards 储存全部牌型，利用 sample 函数随机抽取 13 张牌。使用 for 循环将牌点数加和，最后输出。因不熟悉桥牌，未解决附加题。

4.3 实现代码

```
1 import random
2 suits="CDHS"
3 ranks="23456789TJQKA"
4 points= {'A':4,'K':3,'Q':2,'J':1,'T':0,'9':0,'8':0,'7':0,'6':0,'5':0,'4':0,'3':0,'2':0}
5 #attributes of cards
6 cards=[rank+suit for suit in suits for rank in ranks]
7 givencards=random.sample(cards,13)
8 #deal cards
9 sum=0
10 for card in givencards:
11     sum+=points.get(card[0])
12 #calculate points
13 print("Your cards are "+'.join(givencards))
14 print("Total point is: "+str(sum))
```

4.4 本地测试结果

输出结果如下，点数计算正确。

```
Your cards are 7H QD 2S JH 6H 8C 2C JD AC KC JC QS 6C
Total point is: 14
```